

Experiment No.	1
Experiment Title	Environment Setup & Tooling: Working with Python data science packages ¹
Student Name	Naren Kandasamy
Register Number	3122247001036
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/2026

1 Objective

In this lab, the main goal is to get hands-on experience with standard Python data science libraries like Numpy, Pandas, Scipy, Scikit-learn, and Matplotlib. We want to understand how to handle data arrays, clean up messy datasets, set up basic machine learning pipelines, and create some helpful visualizations.

2 Problem Statement

The main task is to build a basic automated data processing pipeline. We'll look at a few different public datasets (like the Iris dataset, Loan Prediction, Diabetes, Email Spam, and MNIST) to identify what kind of machine learning problem they represent. We also want to walk through the typical workflow steps: loading the data, doing some exploratory data analysis (EDA), cleaning things up, selecting the best features, and finally splitting the data so it's ready for model training later.

3 Dataset Description

Based on the lab requirements, I went through several datasets to figure out what kind of ML task they belong to, what feature selection method works best, and which algorithms we could potentially use:

Dataset	Type of ML task	Feature Selection Technique	Suitable ML Algorithm
Iris Dataset	Supervised (Multi-class Classification)	ANOVA (SelectKBest)	Decision Tree, KNN
Loan Amount Prediction	Supervised (Regression)	F-regression (SelectKBest)	Linear Regression, SVM
Predicting Diabetes	Supervised (Binary Classification)	Chi-square / Mutual Information	Logistic Regression, Random Forest
Classification of Email Spam	Supervised (Binary Classification)	TF-IDF / Chi-square	Naive Bayes, SVM
Handwritten Character Recognition / MNIST	Supervised (Multi-class Classification)	PCA / Variance Threshold	SVM, Neural Networks (CNN)

4 Exploratory Data Analysis

I started by just looking at the datasets using Pandas and Matplotlib to see what we were dealing with. By printing out the first few rows with `df.head()` and running quick statistical summaries via `df.describe()`, I could get a good feel for the numbers. I also checked for missing values and plotted some basic histograms and correlation matrices to see how the features were distributed and if any of them were highly related.

5 Data Preprocessing

To get the data ready for actual machine learning, I built a pipeline that does a few key things:

- **Fixing missing values:** If a number was missing, I usually replaced it with the median. If it was a category (like a word), I just used the most common value (the mode).
- **Encoding:** Since models can't read text strings, I converted categorical variables into numbers using simple label encoding.
- **Scaling features:** I scaled the numerical columns to a standard range using `StandardScaler`. This is super important so that features with naturally larger numbers don't unfairly dominate the distance calculations later.
- **Train-test split:** I split the datasets into a training set and a testing set (typically around an 85%-15% split) using `train_test_split` from Scikit-Learn.
- **Feature selection:** I used `SelectKBest` to figure out which features actually matter the most based on statistical scoring.

6 Machine Learning Algorithm

Not applicable for this introductory experiment.

7 Experimental Setup

Python Version	3.10+
Scikit-Learn Version	1.0+
IDE	Jupyter Notebook / VS Code
Operating System	Linux
Hardware	Standard CPU Workspace

8 Hyperparameter Selection

Not applicable for this introductory experiment.

9 Results

Not applicable for this introductory experiment.

10 Result Visualization

While running the pipeline, I generated a few graphs just to understand the data before modifying it:

- **Class Distribution Histograms:** These showed the imbalance between Spam and Ham in the email dataset, and how many people had diabetes versus those who didn't in the medical data.
- **Missing Value Bar Charts:** These quickly highlighted which columns were missing the most data and needed the most imputation.
- **Correlation Heatmaps:** I used these to check if any numerical features were basically measuring the same thing before I scaled them.

11 Discussion

Going through this lab really showed how useful these Python libraries are for data science. Pandas makes it incredibly easy to fix missing data and move columns around without much code. Scikit-Learn handles all the heavy lifting for scaling and splitting the datasets, which saves a ton of time. I also noticed pretty quickly that different types of data (like tabular numbers versus text data) need completely different cleaning strategies. Overall, the automated pipeline we built does a solid job of preparing diverse datasets for whatever model we want to throw at them next.

12 Conclusion

The lab went really well and the main objectives were met. I got comfortable using Numpy, Pandas, Scikit-Learn, and Matplotlib, and managed to put together a flexible data preprocessing pipeline. Looking at all the different datasets helped me understand how to frame a problem and pick the right approach before we even start training models.

13 References

1. Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825-2830.
2. McKinney, W. "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, 2010.
3. Hunter, J. D. "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering*, vol. 9, no. 3, 2007, pp. 90-95.